WEEK #9 & #10

1# A financial risk analysis tool needs to determine the volatility of a specific stock. Write a Python program that takes a list of daily closing prices and identifies the Maximum (peak) and Minimum (trough) values in that list.

```
prices = list(map(float, input("Enter daily closing prices: ").split()))

if len(prices) == 0:

    print("No prices entered")

else:

    max_price = prices[0]

    min_price = prices[0]

    for price in prices:

        if price > max_price:

            max_price = price

        if price < min_price:

            min_price = price

    print("Maximum price:", max_price)

    print("Minimum price:", min_price)
```

```
Enter daily closing prices: 1 2.5 8 5
Maximum price: 8.0
Minimum price: 1.0
```

2# An inventory checking system allows a store manager to scan a product ID to see if it is currently in stock. Write a Python program to search for a specific element in a list of product IDs. Example 1: • Input: List=[10,20,30], Item=20 • Output: Found Example 2: • Input: List=[10,20,30], Item=40 • Output: Not Found

```
products = list(map(int, input("Enter product IDs: ").split()))

item = int(input("Enter product ID to search: "))

for p in products:

    if p == item:

        print("Found")

        break

else:

    print("Not Found")
```

```
Enter product IDs: 1 2 3 4
Enter product ID to search: 3
Found
```

3# A music playlist app needs to organize songs for the user. Write a Python program to take a raw list of song titles and sort the list alphabetically.

```
songs = input("Enter song titles separated by comma: ").split(",")


for i in range(len(songs)):
    songs[i] = songs[i].strip()
songs.sort()
print("Sorted playlist:", songs)
```

```
Enter song titles separated by comma: reborn,goner,screen
Sorted playlist: ['goner', 'reborn', 'screen']
```

4# An election tally system has a dictionary where keys are candidate names and values are vote counts. Write a Python script to sort this dictionary by value: • First, display the winner (Descending order). • Second, display the candidate with the fewest votes (Ascending order).

```
votes = {}
n = int(input("Enter number of candidates: "))


for i in range(n):
    name = input("Enter candidate name: ")
    count = int(input("Enter vote count: "))
    votes[name] = count
# Convert dictionary to list of tuples
items = list(votes.items())
# Sort descending for winner
items.sort(key=lambda x: x[1], reverse=True)
print("Winner:", items[0])
# Sort ascending for lowest votes
items.sort(key=lambda x: x[1])
print("Fewest votes:", items[0])
```

```
Enter candidate name: Liam
Enter vote count: 45
Enter candidate name: Moki
Enter vote count: 999
Winner: ('Moki', 999)
Fewest votes: ('Liam', 45)
```

5# A Data Science team is compiling a "Complete Patient Record" from three different hospital departments (General Info, Vitals, and Medical History). Since these departments track completely different metrics, there are no overlapping keys. Write a Python script to concatenate these three distinct dictionaries into one new, single Patient_Profile dictionary. Key Point: Concatenation joins distinct pieces of data together. Example: • Input: o Dept_A = {'Name': 'Shivi', 'Age': 23} o Dept_B = {'Height': 165} o Dept_C = {'BloodType': 'B+'} • Output: {'Name': 'Shivi', 'Age': 23, 'Height': 165, 'BloodType': 'B+'}

```
Dept_A = {}

Dept_B = {}

Dept_C = {}

n1 = int(input("Enter number of entries for Dept_A: "))

for i in range(n1):

    key = input("Enter key: ")

    value = input("Enter value: ")

    Dept_A[key] = value

n2 = int(input("Enter number of entries for Dept_B: "))

for i in range(n2):

    key = input("Enter key: ")

    value = input("Enter value: ")

    Dept_B[key] = value

n3 = int(input("Enter number of entries for Dept_C: "))

for i in range(n3):

    key = input("Enter key: ")

    value = input("Enter value: ")

    Dept_C[key] = value

Patient_Profile = {}

# Copy Dept_A

for k in Dept_A:

    Patient_Profile[k] = Dept_A[k]
```

```
# Copy Dept_B
for k in Dept_B:
    Patient_Profile[k] = Dept_B[k]
# Copy Dept_C
for k in Dept_C:
    Patient_Profile[k] = Dept_C[k]
print("Patient Profile:", Patient_Profile)
```

```
Enter number of entries for Dept_A: 1
Enter key: Blood type
Enter value: A+
Enter number of entries for Dept_B: 1
Enter key: Height
Enter value: 4 ft 4
Enter number of entries for Dept_C: 1
Enter key: Gender
Enter value: Male
Patient Profile: {'Blood type': 'A+', 'Height': '4 ft 4 ', 'Gender': 'Male'}
```

6# An embedded system with very limited memory needs to sort a small buffer of data packets. Write a Python script to implement the Optimized Bubble Sort algorithm. (Implement flag mechanism)

```
arr = list(map(int, input("Enter numbers: ").split()))
n = len(arr)
for i in range(n):
    swapped = False
    for j in range(0, n-i-1):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
            swapped = True

    if not swapped:
        break

print("Sorted list:", arr)
```

```
Enter numbers: 5 6 3 2 5 7 213 55
Sorted list: [2, 3, 5, 5, 6, 7, 55, 213]
```

7# A Big Data processing engine needs to sort a massive log file efficiently. Write a Python program to implement the Merge Sort algorithm. You must use recursion to demonstrate the divide-and-conquer methodology. Explain its best, average, and worst-case complexities.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    while i < len(left):
        result.append(left[i])
        i += 1
    while j < len(right):
        result.append(right[j])
        j += 1
    return result
```

```
Enter numbers: 5 6 3 2 5 7 213 55
Sorted list: [2, 3, 5, 5, 6, 7, 55, 213]
```

8# A real-time gaming engine needs a high-speed sorting algorithm for rendering objects. Write a Python program to implement the Quick Sort algorithm. Explain its best, average, and worst-case complexities.

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]

    left = []
    middle = []
    right = []

    for x in arr:
        if x < pivot:
            left.append(x)
        elif x == pivot:
            middle.append(x)
        else:
            right.append(x)

    return quick_sort(left) + middle + quick_sort(right)

nums = list(map(int, input("Enter numbers: ").split()))
print("Sorted list:", quick_sort(nums))
```

```
Enter numbers: 3  5 67 4 2 1
Sorted list: [1, 2, 3, 4, 5, 67]
```